AI-Based Non-Contact Vital Sign Monitoring Using rPPG for Sustainable Healthcare

---

Overview

This project presents an AI-based system for non-contact monitoring of vital signs using Remote Photoplethysmography (rPPG). The system estimates Heart Rate (HR), Respiratory Rate (RR), and SpO_2 from facial video captured using a standard RGB camera.

Traditional monitoring systems rely on contact-based sensors such as ECG and pulse oximeters, which can be uncomfortable and unhygienic. This project provides a contactless, real-time, and scalable solution by combining computer vision and deep learning techniques.

The system captures facial video, extracts subtle skin color variations caused by blood flow, and processes them using a CNN–LSTM model to predict vital signs.

Features

Module 1: Video Acquisition

- Captures real-time facial video using webcam
- Provides continuous input for analysis

Module 2: Face Detection & Tracking

- Detects face using computer vision techniques
- Tracks face across frames to maintain consistency

Module 3: ROI Extraction

- Extracts Regions of Interest (forehead & cheeks)
- Focuses on areas with strong rPPG signals

Module 4: Preprocessing

- Noise removal and normalization
- Reduces lighting and motion effects

Module 5: Feature Extraction

- Extracts rPPG signals from pixel intensity changes
- Converts video data into physiological signals

Module 6: CNN–LSTM Model

- CNN captures spatial features
- LSTM captures temporal patterns
- Learns heart rate and respiration patterns

Module 7: Prediction & Output

- Predicts HR, RR, and SpO_2
- Displays results in real-time

Module 8: Frontend Interface

- Live camera feed
- Dashboard showing vital signs
- User-friendly UI

Dataset

- Dataset Used: Vital Videos Dataset
- Total Files: 293
- Videos: 193 (.mp4)
- Labels: 99 (.json)

The dataset includes variations in:

- Lighting conditions
- Subject motion
- Facial orientation

This helps improve model robustness.

Tools & Technologies

Programming Language

- Python

Libraries & Frameworks

- OpenCV – Face detection & video processing
- NumPy, Pandas – Data handling
- TensorFlow / Keras – Deep learning
- Matplotlib – Visualization

Core Concepts

- Computer Vision
- Deep Learning (CNN + LSTM)
- Signal Processing (rPPG)
- Time Series Analysis

Requirements

Software Requirements

- Python (3.x)
- Web Browser (Chrome/Edge)

Hardware Requirements

- System with webcam
- Minimum 8GB RAM (recommended)
- GPU (optional, for faster training)

Development Tools

- VS Code
- Jupyter Notebook

Methodology

The system follows a pipeline approach:

1. Capture facial video using RGB camera
2. Detect and track face in each frame
3. Extract ROI (forehead & cheeks)
4. Preprocess frames (normalize, denoise)
5. Extract rPPG signals from pixel variations
6. Feed data into CNN–LSTM model
7. Predict HR, RR, and SpO_2
8. Display results in real-time interface

The CNN extracts spatial features, while LSTM captures temporal dependencies, enabling accurate physiological signal estimation.

Results

- Accuracy: ~98%
- Mean Absolute Error: ~2 BPM
- Strong correlation between actual and predicted values

Observations:

- Works best under stable lighting
- Slight deviations with motion or illumination changes
- Real-time predictions are consistent and reliable

Getting Started

1. Clone the Repository

```
git clone https://github.com/pranathii15/non-contact-vital-sign-monitoring-rppg
cd frontend
```

2. Install Dependencies

Ensure that Node.js (version 16 or above) is installed, then run:

```
npm install
```

3. Configure Environment Variables

Create a file named `.env.local` inside the `frontend/` directory and add the following:

EMAIL_USER=your_email@gmail.com
EMAIL_PASS=your_app_password

TWILIO_SID=your_account_sid
TWILIO_AUTH=your_auth_token
TWILIO_PHONE=your_twilio_number

Notes:

- The above information is private and should not be shared.
- Use a Gmail App Password instead of your regular email password.
- Obtain Twilio credentials from your Twilio dashboard.

4. Run the Application

Start the development server using:

```
npm run dev
```

5. Access the Application

Open a web browser and navigate to:

http://localhost:3000

6. Using the System

- Navigate to the dashboard page.
- Allow camera access when prompted.

- Position yourself in front of the webcam.
- The system will begin monitoring and displaying:
    - Heart Rate (HR)
    - SpO_2
    - Respiratory Rate (RR)
- Alerts will be automatically triggered if abnormal values are detected.

7. Testing Alerts

- Ensure that environment variables are correctly configured.
- Trigger abnormal values (if supported) to verify:
    - Email notifications
    - SMS alerts

8. Troubleshooting

- If the camera does not work, check browser permissions and ensure you are using localhost or HTTPS.
- If email notifications fail, verify the Gmail App Password and configuration.
- If SMS alerts fail, check Twilio credentials and account balance.

9. Stop the Server

To stop the application, press:

CTRL + C

Notes

- No separate backend setup is required since API routes are handled within Next.js.
- A stable internet connection is required for email and SMS services.
- The system performs best under stable lighting conditions with minimal movement.

Future Work

- Improve robustness against motion artifacts
- Enhance SpO_2 prediction accuracy
- Integrate infrared/depth sensors
- Develop mobile/web application
- Add Explainable AI for better interpretation
- Expand dataset for better generalization